Design Templates

Generic Template — Format Guide & Test Harness

Every template file in this folder follows the same two-part structure:

1. **Template algorithm + template code** — the reusable pattern, with specific LeetCode problems used as concrete examples inside the code.
2. **Quick reference table + question/solution** — a table listing every problem (with a *Variation from Template* column), followed by each problem's full solution where deviations are marked `// ← VARIATION: explanation`.

Generic Test Harness

Use this boilerplate to run any single problem locally. Replace `data` / `solve()` with the problem's input and logic.

```
class Question {

    private int data;

    public Question(int data) {
        this.data = data;
    }

    public void solve() {
        // TODO
    }
}

public class Solution {

    public static void main(String[] args) {
        Question q = new Question(0);
        q.solve();
    }
}
```

Naming Conventions (enforced across all template files)

Context	Convention
Index variables	`i`, `j`, `k` (never `left`, `right`, `low`, `mid`, `high`)
Linked list	`sentinel`, `previous`, `current` (never `prev`, `cur`, `curr`)
String building	`StringBuilder builder = new StringBuilder();` (never `sb`)
Result accumulator	`result` (never `res`); use `output` if `result` is already taken in scope
Queue / deque	`Queue<Integer> queue = new ArrayDeque<>();` (never `dq`)
BFS over tree	`Queue<TreeNode> queue = new ArrayDeque<>();`
Grid directions	`int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};` (DOWN, UP, RIGHT, LEFT); iterate `for (int dir=0; dir<4; dir++)` using `i+dr[dir]`, `j+dc[dir]`
Character frequency	`count[ch - 'a']++;`
Digit frequency	`count[ch - '0']++;`
Char to integer build	`num = num * 10 + (ch - '0');`

Template Index

File	Category	Core Pattern
`binary_search.md`	Binary Search	Closed `[i,j]`, half-open, minimize/maximize answer space
`two_pointers.md`	Two Pointers	Opposite ends, same-direction write pointer, Dutch flag
`prefix_sum_hashmap.md`	Prefix Sum	Count/length with HashMap, tree path sums
`sorting.md`	Sorting	Merge sort, quick sort, quick select, count-during-merge
`sliding_window.md`	Sliding Window	Fixed, max-variable, min-variable windows
`linked_list.md`	Linked List	Delete/filter, reversal, fast/slow, merge, composite
`dfs.md`	DFS / Backtracking	Tree return-up/pass-down, grid DFS, graph 3-color, backtracking, BST
`bfs.md`	BFS	Level-order with size snapshot, position indexing
`graph.md`	Graph	BFS shortest path, topo sort, union find, Dijkstra, bipartite, Prim's MST
`dynamic_programming.md`	Dynamic Programming	1D/2D/grid/interval DP, knapsack, state machine
`stack_queue.md`	Stack / Queue / Heap	Monotone stack/deque, two heaps, priority queue
`greedy.md`	Greedy	Sort-by-end, sort-by-start, sort + heap, non-interval
`string.md`	String	Frequency counting, hashing, bucket sort, expand-from-center
`bit_manipulation.md`	Bit Manipulation	XOR cancel, mod-k state machine, bitmask as set, power checks
`trie.md`	Trie	Insert/search/startsWith, wildcard DFS, grid pruning
`design.md`	Design	HashMap + linked list (LRU), frequency maps (LFU), array + map
`math.md`	Math	Fast power, base conversion, sieve, digit manipulation

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
146	LRU Cache	Design a data structure that follows the LRU (Least Recently Used) eviction policy. `get(key)` returns the value or -1. `put(key, value)` inserts/updates; evicts LRU when over capacity. Both operations must be O(1).	the HashMap answers "where is this key" instantly while the linked list keeps everything ordered by recency, so the victim to evict is always the node next to the tail.	O(1) get/put	O(capacity)	Standard
460	LFU Cache	Design a data structure for a Least Frequently Used cache. Both `get` and `put` must be O(1). On eviction, remove the least frequently used key; among ties in frequency, remove the least recently used.	bucket keys by how often they're used; the eviction victim is always the oldest key in the lowest-frequency bucket, which `minFreq` points straight at.	O(1) all operations	O(capacity)	frequency tracking on every access
380	Insert Delete GetRandom O(1)	Design a set where `insert`, `remove`, and `getRandom` all run in O(1) average time. `getRandom` returns any element with equal probability.	an array gives O(1) random indexing but O(n) middle-deletion; swapping the victim with the last element turns deletion into an O(1) pop while staying dense.	O(1) all ops	O(n)	swap target with last element
170	Two Sum III - Data structure design	Design a data structure with `add(number)` to store a number and `find(value)` returning true if any pair of stored numbers sums to `value`.	the complement check from classic Two Sum, but persisted across calls; the frequency map lets a single number satisfy a pair only when it appears at least twice.	O(1) add, O(n) find	O(n)	Standard
173	Binary Search Tree Iterator	Implement an iterator over a BST: `next()` returns the next smallest element and `hasNext()` reports whether one exists. Use O(h) memory.	an in-order traversal paused mid-flight — the stack stores exactly the unvisited ancestors, so memory stays proportional to tree height.	O(1) amortized next/hasNext	O(h)	Standard
244	Shortest Word Distance II	Initialize with a word list. `shortest(word1, word2)` returns the minimum index distance between the two words, supporting repeated queries efficiently.	because both index lists are sorted, the closest pair is found by always advancing the pointer at the smaller index — the same merge step as in sorted-list intersection.	O(n) init, O(a+b) per query	O(n)	Standard
245	Shortest Word Distance III	Given a word list and two words (which may be equal), return the shortest index distance. When `word1 == word2`, find the closest pair of distinct occurrences of that word.	one scan suffices for a single query; the equal-word case is just "closest pair of repeated occurrences," handled by remembering the previous match.	O(n)	O(1)	equal words use previous i
251	Flatten 2D Vector	Design an iterator over a 2D vector with `next()` and `hasNext()` that yields all elements in row-major order.	keep a `(row, col)` cursor and lazily skip empty rows; the skip logic concentrated in one helper keeps both methods correct.	O(1) amortized per call	O(1)	Standard
281	Zigzag Iterator	Given two lists, design an iterator that returns elements alternating between them; when one is exhausted, continue with the other.	a round-robin queue of iterators rotates through the lists; finished iterators simply fall out of the queue.	O(1) per call	O(k)	Standard
341	Flatten Nested List Iterator	Implement an iterator that flattens a nested list of integers, given the `NestedInteger` interface (`isInteger()`, `getInteger()`, `getList()`).	a stack performs the depth-first unwrap lazily — flattening happens only as far as the next requested integer.	O(1) amortized next, O(n) total	O(n)	Standard
346	Moving Average from Data Stream	Given a window size, design `next(val)` that returns the moving average of the last `size` values in the stream.	the running sum avoids re-summing the window; the queue only tracks which value leaves next.	O(1) per call	O(size)	Standard
348	Design Tic-Tac-Toe	Design an `n x n` Tic-Tac-Toe game. `move(row, col, player)` returns the winning player (1 or 2) if that move wins, else 0. Detect a winner in O(1) per move.	signing the contributions lets a single counter detect either player — only checking the four lines that the current move touches keeps it O(1).	O(1) per move	O(n)	Standard
381	Insert Delete GetRandom O(1) - Duplicates allowed	Like #380 but duplicates are allowed. `insert` returns true if the value was not already present, `remove` removes one occurrence, and `getRandom` picks proportionally to multiplicity.	the swap-with-last trick from #380 extends to duplicates by storing a set of indices per value instead of a single index. ← VARIATION: value → set of indices.	O(1) average all ops	O(n)	swap target with last
519	Random Flip Matrix	Design a structure over an `m x n` grid of zeros. `flip()` randomly picks a remaining 0-cell, sets it to 1, and returns its `[row, col]`. `reset()` restores all zeros.	virtual swap-to-end on a 1D index space gives uniform O(1) flips without materializing the full grid — the map only records cells that were moved.	O(1) flip, O(1) reset	O(flips between resets)	Standard
622	Design Circular Queue	Design a fixed-capacity circular queue with `enQueue`, `deQueue`, `Front`, `Rear`, `isEmpty`, `isFull`.	tracking `head` and `count` (rather than head/tail pointers) avoids the empty-vs-full ambiguity entirely.	O(1) all ops	O(k)	Standard
641	Design Circular Deque	Design a double-ended circular queue with `insertFront`, `insertLast`, `deleteFront`, `deleteLast`, `getFront`, `getRear`, `isEmpty`, `isFull`.	the #622 head/count model extends to both ends — inserting at the front just rotates `head` backward.	O(1) all ops	O(k)	rotate head backward
705	Design HashSet	Design a HashSet without built-in hash libraries, supporting `add`, `remove`, and `contains`.	separate chaining is the textbook collision strategy; a fixed prime-ish bucket count keeps chains short for typical inputs.	O(1) average per op	O(n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
706	Design HashMap	Design a HashMap without built-in hash libraries, supporting `put`, `get`, and `remove`.	identical structure to #705 but each node carries a value, so collisions are resolved by scanning a short chain of entries.	O(1) average per op	O(n)	update existing value
707	Design Linked List	Design a linked list supporting `get`, `addAtHead`, `addAtTail`, `addAtIndex`, and `deleteAtIndex`.	sentinels remove all null-edge cases for head/tail insertion, so every operation is the same splice on interior nodes.	O(index) get/add/delete	O(n)	Standard
729	My Calendar I	Implement `book(start, end)` for a half-open interval `[start, end)`; return true and add the booking if it does not overlap any existing one, else false.	the sorted map narrows the conflict check to at most two neighbors, giving O(log n) per booking.	O(log n) per book	O(n)	Standard
731	My Calendar II	Implement `book(start, end)`; return true unless adding it would cause a triple booking (three events overlapping at the same instant).	the difference array turns "max concurrent events" into a prefix-sum scan, and rolling back keeps the structure clean on rejection.	O(n) per book	O(n)	triple booking detected, roll back
732	My Calendar III	Implement `book(start, end)` returning the maximum `k` such that some instant is covered by `k` events (the largest k-booking so far).	the same difference-array sweep as #731, but here we report the peak concurrency rather than reject it.	O(n) per book	O(n)	track peak overlap
933	Number of Recent Calls	Implement `ping(t)` (calls arrive with strictly increasing `t`) returning how many pings occurred in the inclusive window `[t - 3000, t]`.	because times arrive sorted, expired pings are always at the front — a sliding window over a queue.	O(1) amortized per ping	O(window size)	Standard
1570	Dot Product of Two Sparse Vectors	Design a `SparseVector` initialized from an array; `dotProduct(other)` returns the dot product, taking advantage of sparsity.	since most entries are zero, storing and iterating only nonzeros makes the cost proportional to the number of nonzero terms, not the vector length.	O(n) init, O(min nonzeros) per dot product	O(nonzeros)	iterate fewer entries

Canonical Template — HashMap + Doubly Linked List (LRU)

```
// MENTAL MODEL: HashMap gives O(1) lookup; the doubly linked list orders by recency so the LRU is always one hop from tail.
// WHEN: "O(1) get/put with eviction by recency or frequency"

// CacheNode for doubly linked list (named CacheNode to distinguish from ListNode contexts)
class CacheNode {
    int key, value;
    CacheNode previous, next;
    CacheNode(int key, int value) { this.key = key; this.value = value; }
}

// ── SENTINEL NODES ── head (MRU side) ↔ ... ↔ tail (LRU side)
CacheNode head = new CacheNode(0, 0), tail = new CacheNode(0, 0);
// Initialize: head <-> tail
head.next = tail; tail.previous = head;

// ── REMOVE NODE ──
void remove(CacheNode node) {
    node.previous.next = node.next;
    node.next.previous = node.previous;
}

// ── INSERT AFTER HEAD (= mark as MRU) ──
void insertAfterHead(CacheNode node) {
    node.next = head.next;
    node.previous = head;
    head.next.previous = node;
    head.next = node;
}

// ── LRU EVICTION ── tail.previous is LRU
CacheNode lru = tail.previous;
remove(lru);
map.remove(lru.key);
```

Variations

```
// VARIATION 1: LFU — adds a frequency dimension
// Need: key→value, key→freq, freq→orderedSet<key>, and minFreq
// When a key is accessed: freq++, move from freqToKeys[oldFreq] to freqToKeys[newFreq]
// On eviction: remove first element from freqToKeys[minFreq]
// LinkedHashSet preserves insertion order → oldest at front for same frequency

// VARIATION 2: Insert Delete GetRandom — use array for O(1) random
// ArrayList stores values; HashMap stores val→index in array
// On remove: swap target with last element, update map, remove last
// On getRandom: return list.get(random.nextInt(list.size()))
```

Part 1 — LRU Cache

#146 LRU Cache

Description: Design a data structure that follows the LRU (Least Recently Used) eviction policy. `get(key)` returns the value or -1. `put(key, value)` inserts/updates; evicts LRU when over capacity. Both operations must be O(1).

Algorithm: Combine a HashMap (O(1) key lookup) with a doubly linked list (O(1) insertion/deletion). The list order represents recency — most recently used at head, least recently used at tail. On every `get` or `put`, move the accessed node to the front.

Intuition: the HashMap answers "where is this key" instantly while the linked list keeps everything ordered by recency, so the victim to evict is always the node next to the tail.

```
class LRUCache {
    private final int capacity;
    private final Map<Integer, CacheNode> map = new HashMap<>();
    private final CacheNode head = new CacheNode(0, 0), tail = new CacheNode(0, 0);

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.previous = head;
    }

    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        CacheNode node = map.get(key);
        remove(node);
        insertAfterHead(node);          // ← move to MRU position
        return node.value;
    }

    public void put(int key, int value) {
        if (map.containsKey(key)) {
            CacheNode node = map.get(key);
            node.value = value;
            remove(node);
            insertAfterHead(node);      // ← update and move to MRU position
        } else {
            if (map.size() == capacity) {
                CacheNode lru = tail.previous;   // ← LRU is just before tail sentinel
                remove(lru);
                map.remove(lru.key);
            }
            CacheNode node = new CacheNode(key, value);
            map.put(key, node);
            insertAfterHead(node);
        }
    }

    private void remove(CacheNode node) {
        node.previous.next = node.next;
        node.next.previous = node.previous;
    }

    private void insertAfterHead(CacheNode node) {
        node.next = head.next;
        node.previous = head;
        head.next.previous = node;
        head.next = node;
    }

    static class CacheNode {
        int key, value;
        CacheNode previous, next;
        CacheNode(int key, int value) { this.key = key; this.value = value; }
    }
}
```

Time O(1) get/put | **Space** O(capacity)

Part 2 — LFU Cache

#460 LFU Cache

Description: Design a data structure for a Least Frequently Used cache. Both `get` and `put` must be O(1). On eviction, remove the least frequently used key; among ties in frequency, remove the least recently used.

Algorithm: Three maps: `keyToVal`, `keyToFreq`, `freqToKeys` (freq → LinkedHashSet for insertion-order LRU within same freq). Track `minFreq`. On access: increment freq, move key from old to new freq set; if old freq set is now empty and it was minFreq, increment minFreq.

Intuition: bucket keys by how often they're used; the eviction victim is always the oldest key in the lowest-frequency bucket, which `minFreq` points straight at.

```
class LFUCache {
    private final int capacity;
    private int minFreq = 0;
    private final Map<Integer, Integer> keyToVal  = new HashMap<>();
    private final Map<Integer, Integer> keyToFreq = new HashMap<>();
    private final Map<Integer, LinkedHashSet<Integer>> freqToKeys = new HashMap<>();

    public LFUCache(int capacity) { this.capacity = capacity; }

    public int get(int key) {
        if (!keyToVal.containsKey(key)) return -1;
        increaseFreq(key);              // ← VARIATION: frequency tracking on every access
        return keyToVal.get(key);
    }

    public void put(int key, int value) {
        if (capacity <= 0) return;
        if (keyToVal.containsKey(key)) {
            keyToVal.put(key, value);
            increaseFreq(key);
        } else {
            if (keyToVal.size() >= capacity)
                removeMinFreqKey();     // ← VARIATION: evict by min frequency
            keyToVal.put(key, value);
            keyToFreq.put(key, 1);
            freqToKeys.computeIfAbsent(1, k -> new LinkedHashSet<>()).add(key);
            minFreq = 1;                // ← VARIATION: reset minFreq to 1 on new insert
        }
    }

    private void increaseFreq(int key) {
        int freq = keyToFreq.get(key);
        keyToFreq.put(key, freq + 1);
        freqToKeys.get(freq).remove(key);
        if (freqToKeys.get(freq).isEmpty()) {
            freqToKeys.remove(freq);
            if (minFreq == freq) minFreq++;   // ← VARIATION: update minFreq only if this was min
        }
        freqToKeys.computeIfAbsent(freq + 1, k -> new LinkedHashSet<>()).add(key);
    }

    private void removeMinFreqKey() {
        LinkedHashSet<Integer> keys = freqToKeys.get(minFreq);
        int evict = keys.iterator().next();   // ← LinkedHashSet: first = oldest at this freq
        keys.remove(evict);
        if (keys.isEmpty()) freqToKeys.remove(minFreq);
        keyToVal.remove(evict);
        keyToFreq.remove(evict);
    }
}
```

Time O(1) all operations | **Space** O(capacity)

Part 3 — Array + HashMap (Insert Delete GetRandom)

#380 Insert Delete GetRandom O(1)

Description: Design a set where `insert`, `remove`, and `getRandom` all run in O(1) average time. `getRandom` returns any element with equal probability.

Algorithm: HashMap maps value→index in an ArrayList. On remove, swap the target with the last element (update the swapped element's index in the map), then remove the last. This keeps the array dense without shifting. `getRandom` uses `Random.nextInt(size)`.

Intuition: an array gives O(1) random indexing but O(n) middle-deletion; swapping the victim with the last element turns deletion into an O(1) pop while staying dense.

```
class RandomizedSet {
    private final Map<Integer, Integer> map = new HashMap<>();  // val → index in list
    private final List<Integer> list = new ArrayList<>();
    private final Random rand = new Random();

    public boolean insert(int val) {
        if (map.containsKey(val)) return false;
        list.add(val);
        map.put(val, list.size() - 1);
        return true;
    }

    public boolean remove(int val) {
        if (!map.containsKey(val)) return false;
        int i = map.get(val);
        int last = list.get(list.size() - 1);
        list.set(i, last);              // ← VARIATION: swap target with last element
        map.put(last, i);              // ← VARIATION: update swapped element's index
        list.remove(list.size() - 1);
        map.remove(val);
        return true;
    }

    public int getRandom() {
        return list.get(rand.nextInt(list.size()));  // ← VARIATION: O(1) random via ArrayList
    }
}
```

Time O(1) all ops | **Space** O(n)

LRU vs LFU vs RandomizedSet — Pattern Chooser

Signal	Design
"evict least **recently** used"	LRU: HashMap + doubly linked list, move to head on access
"evict least **frequently** used"	LFU: 3 HashMaps + LinkedHashSet + `minFreq` tracking
"O(1) insert/delete/random"	RandomizedSet: HashMap + ArrayList, swap-with-last on delete

Key Invariants

```
LRU:  head.next = MRU,  tail.previous = LRU
LFU:  minFreq always points to the smallest existing frequency bucket
      LinkedHashSet preserves insertion order → iterator().next() = LRU within bucket
RandomizedSet: list[map[val]] == val at all times (after swap, update the map for the swapped key)
```

Additional Reference Problems

#170 Two Sum III - Data structure design

Description: Design a data structure with `add(number)` to store a number and `find(value)` returning true if any pair of stored numbers sums to `value`.

Algorithm: Keep a frequency map of stored numbers. `find` iterates the keys; for each `key` check whether `value − key` exists — if it equals `key`, require count ≥ 2.

Intuition: the complement check from classic Two Sum, but persisted across calls; the frequency map lets a single number satisfy a pair only when it appears at least twice.

```
class TwoSum {
    private final Map<Integer, Integer> map = new HashMap<>();

    public TwoSum() {
    }

    public void add(int number) {
        map.merge(number, 1, Integer::sum);
    }

    public boolean find(int value) {
        for (int key : map.keySet()) {
            int complement = value - key;
            if (complement == key) {
                if (map.get(key) >= 2) {
                    return true;
                }
            } else if (map.containsKey(complement)) {
                return true;
            }
        }
        return false;
    }
}
```

Time O(1) add, O(n) find | **Space** O(n)

#173 Binary Search Tree Iterator

Description: Implement an iterator over a BST: `next()` returns the next smallest element and `hasNext()` reports whether one exists. Use O(h) memory.

Algorithm: Maintain a stack holding the path of left descendants. Push all left children from the root initially; `next` pops a node, pushes the left spine of its right child.

Intuition: an in-order traversal paused mid-flight — the stack stores exactly the unvisited ancestors, so memory stays proportional to tree height.

```
class BSTIterator {
    private final Deque<TreeNode> stack = new ArrayDeque<>();

    public BSTIterator(TreeNode root) {
        pushLeft(root);
    }

    public int next() {
        TreeNode node = stack.pop();
        pushLeft(node.right);
        return node.val;
    }

    public boolean hasNext() {
        return !stack.isEmpty();
    }

    private void pushLeft(TreeNode node) {
        while (node != null) {
            stack.push(node);
            node = node.left;
        }
    }

    static class TreeNode {
        int val;
        TreeNode left, right;
        TreeNode(int val) { this.val = val; }
    }
}
```

Time O(1) amortized next/hasNext | **Space** O(h)

#244 Shortest Word Distance II

Description: Initialize with a word list. `shortest(word1, word2)` returns the minimum index distance between the two words, supporting repeated queries efficiently.

Algorithm: Precompute a map from each word to its sorted list of indices. For a query, merge-walk the two index lists with two pointers, advancing the smaller index to minimize the gap.

Intuition: because both index lists are sorted, the closest pair is found by always advancing the pointer at the smaller index — the same merge step as in sorted-list intersection.

```
class WordDistance {
    private final Map<String, List<Integer>> map = new HashMap<>();

    public WordDistance(String[] wordsDict) {
        for (int i = 0; i < wordsDict.length; i++) {
            map.computeIfAbsent(wordsDict[i], k -> new ArrayList<>()).add(i);
        }
    }

    public int shortest(String word1, String word2) {
        List<Integer> first = map.get(word1);
        List<Integer> second = map.get(word2);
        int i = 0, j = 0, result = Integer.MAX_VALUE;
        while (i < first.size() && j < second.size()) {
            int a = first.get(i), b = second.get(j);
            result = Math.min(result, Math.abs(a - b));
            if (a < b) {
                i++;
            } else {
                j++;
            }
        }
        return result;
    }
}
```

Time O(n) init, O(a+b) per query | **Space** O(n)

#245 Shortest Word Distance III

Description: Given a word list and two words (which may be equal), return the shortest index distance. When `word1 == word2`, find the closest pair of distinct occurrences of that word.

Algorithm: Single pass tracking the last seen index of each word. When the two words are equal, reuse a single previous-index variable and compare consecutive occurrences.

Intuition: one scan suffices for a single query; the equal-word case is just "closest pair of repeated occurrences," handled by remembering the previous match.

```
class Solution {
    public int shortestWordDistance(String[] wordsDict, String word1, String word2) {
        boolean same = word1.equals(word2);
        int i = -1, j = -1, result = Integer.MAX_VALUE;
        for (int k = 0; k < wordsDict.length; k++) {
            String word = wordsDict[k];
            if (word.equals(word1)) {
                if (same) {                       // ← VARIATION: equal words use previous i
                    if (i >= 0) {
                        result = Math.min(result, k - i);
                    }
                    i = k;
                } else {
                    i = k;
                    if (j >= 0) {
                        result = Math.min(result, Math.abs(i - j));
                    }
                }
            } else if (word.equals(word2)) {
                j = k;
                if (i >= 0) {
                    result = Math.min(result, Math.abs(i - j));
                }
            }
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

#251 Flatten 2D Vector

Description: Design an iterator over a 2D vector with `next()` and `hasNext()` that yields all elements in row-major order.

Algorithm: Track an outer row index and inner column index. Before reading, advance past exhausted/empty rows so the cursor always points at a valid element when one exists.

Intuition: keep a `(row, col)` cursor and lazily skip empty rows; the skip logic concentrated in one helper keeps both methods correct.

```
class Vector2D {
    private final int[][] vec;
    private int row = 0, col = 0;

    public Vector2D(int[][] vec) {
        this.vec = vec;
    }

    public int next() {
        advance();
        return vec[row][col++];
    }

    public boolean hasNext() {
        advance();
        return row < vec.length;
    }

    private void advance() {
        while (row < vec.length && col == vec[row].length) {
            row++;
            col = 0;
        }
    }
}
```

Time O(1) amortized per call | **Space** O(1)

#281 Zigzag Iterator

Description: Given two lists, design an iterator that returns elements alternating between them; when one is exhausted, continue with the other.

Algorithm: Hold a queue of per-list iterators. On `next`, poll an iterator, take its value, and re-enqueue it if it still has elements — naturally generalizing to k lists.

Intuition: a round-robin queue of iterators rotates through the lists; finished iterators simply fall out of the queue.

```
class ZigzagIterator {
    private final Queue<Iterator<Integer>> queue = new ArrayDeque<>();

    public ZigzagIterator(List<Integer> v1, List<Integer> v2) {
        if (!v1.isEmpty()) {
            queue.offer(v1.iterator());
        }
        if (!v2.isEmpty()) {
            queue.offer(v2.iterator());
        }
    }

    public int next() {
        Iterator<Integer> iterator = queue.poll();
        int value = iterator.next();
        if (iterator.hasNext()) {
            queue.offer(iterator);          // ← re-enqueue for round-robin
        }
        return value;
    }

    public boolean hasNext() {
        return !queue.isEmpty();
    }
}
```

Time O(1) per call | **Space** O(k)

#341 Flatten Nested List Iterator

Description: Implement an iterator that flattens a nested list of integers, given the `NestedInteger` interface (`isInteger()`, `getInteger()`, `getList()`).

Algorithm: Push the list onto a stack in reverse. On each `hasNext`, peek the top; while it is a list, pop and push its contents in reverse so an integer is exposed at the top.

Intuition: a stack performs the depth-first unwrap lazily — flattening happens only as far as the next requested integer.

```
class NestedIterator implements Iterator<Integer> {
    private final Deque<NestedInteger> stack = new ArrayDeque<>();

    public NestedIterator(List<NestedInteger> nestedList) {
        pushReversed(nestedList);
    }

    @Override
    public Integer next() {
        return stack.pop().getInteger();
    }

    @Override
    public boolean hasNext() {
        while (!stack.isEmpty() && !stack.peek().isInteger()) {
            List<NestedInteger> list = stack.pop().getList();
            pushReversed(list);
        }
        return !stack.isEmpty();
    }

    private void pushReversed(List<NestedInteger> list) {
        for (int i = list.size() - 1; i >= 0; i--) {
            stack.push(list.get(i));
        }
    }

    interface NestedInteger {
        boolean isInteger();
        Integer getInteger();
        List<NestedInteger> getList();
    }
}
```

Time O(1) amortized next, O(n) total | **Space** O(n)

#346 Moving Average from Data Stream

Description: Given a window size, design `next(val)` that returns the moving average of the last `size` values in the stream.

Algorithm: Keep a fixed-size queue and a running sum. On each value, add it and subtract the evicted element when the window overflows.

Intuition: the running sum avoids re-summing the window; the queue only tracks which value leaves next.

```
class MovingAverage {
    private final Queue<Integer> queue = new ArrayDeque<>();
    private final int size;
    private double sum = 0;

    public MovingAverage(int size) {
        this.size = size;
    }

    public double next(int val) {
        queue.offer(val);
        sum += val;
        if (queue.size() > size) {
            sum -= queue.poll();
        }
        return sum / queue.size();
    }
}
```

Time O(1) per call | **Space** O(size)

#348 Design Tic-Tac-Toe

Description: Design an `n x n` Tic-Tac-Toe game. `move(row, col, player)` returns the winning player (1 or 2) if that move wins, else 0. Detect a winner in O(1) per move.

Algorithm: Maintain per-row and per-column counters plus two diagonal counters. Encode player 1 as +1 and player 2 as -1; a line reaches magnitude `n` exactly when one player fills it.

Intuition: signing the contributions lets a single counter detect either player — only checking the four lines that the current move touches keeps it O(1).

```
class TicTacToe {
    private final int[] rows;
    private final int[] cols;
    private int diagonal = 0, antiDiagonal = 0;
    private final int n;

    public TicTacToe(int n) {
        this.n = n;
        rows = new int[n];
        cols = new int[n];
    }

    public int move(int row, int col, int player) {
        int delta = (player == 1) ? 1 : -1;
        rows[row] += delta;
        cols[col] += delta;
        if (row == col) {
            diagonal += delta;
        }
        if (row + col == n - 1) {
            antiDiagonal += delta;
        }
        if (Math.abs(rows[row]) == n || Math.abs(cols[col]) == n
                || Math.abs(diagonal) == n || Math.abs(antiDiagonal) == n) {
            return player;
        }
        return 0;
    }
}
```

Time O(1) per move | **Space** O(n)

#381 Insert Delete GetRandom O(1) - Duplicates allowed

Description: Like #380 but duplicates are allowed. `insert` returns true if the value was not already present, `remove` removes one occurrence, and `getRandom` picks proportionally to multiplicity.

Algorithm: ArrayList stores all values; a map from value to a set of its indices. On remove, swap one occurrence with the last element and fix both index sets.

Intuition: the swap-with-last trick from #380 extends to duplicates by storing a set of indices per value instead of a single index. ← VARIATION: value → set of indices.

```
class RandomizedCollection {
    private final List<Integer> list = new ArrayList<>();
    private final Map<Integer, Set<Integer>> indices = new HashMap<>();
    private final Random rand = new Random();

    public boolean insert(int val) {
        boolean absent = !indices.containsKey(val) || indices.get(val).isEmpty();
        indices.computeIfAbsent(val, k -> new HashSet<>()).add(list.size());
        list.add(val);
        return absent;
    }

    public boolean remove(int val) {
        Set<Integer> set = indices.get(val);
        if (set == null || set.isEmpty()) {
            return false;
        }
        int i = set.iterator().next();
        set.remove(i);
        int lastIndex = list.size() - 1;
        int last = list.get(lastIndex);
        list.set(i, last);                      // ← VARIATION: swap target with last
        if (i != lastIndex) {
            Set<Integer> lastSet = indices.get(last);
            lastSet.remove(lastIndex);
            lastSet.add(i);                     // ← VARIATION: fix moved element's index
        }
        list.remove(lastIndex);
        return true;
    }

    public int getRandom() {
        return list.get(rand.nextInt(list.size()));
    }
}
```

Time O(1) average all ops | **Space** O(n)

#519 Random Flip Matrix

Description: Design a structure over an `m x n` grid of zeros. `flip()` randomly picks a remaining 0-cell, sets it to 1, and returns its `[row, col]`. `reset()` restores all zeros.

Algorithm: Treat the grid as a flat range `[0, total)`. Maintain a map from a chosen flat index to the value currently occupying the "tail" slot. Pick a random index in the shrinking range and swap-with-last via the map (Fisher–Yates style).

Intuition: virtual swap-to-end on a 1D index space gives uniform O(1) flips without materializing the full grid — the map only records cells that were moved.

```java
class Solution {
    private final int rows, cols;
    private int remaining;
    private final Map<Integer, Integer> map = new HashMap<>();
    private final Random rand = new Random();

    public Solution(int m, int n) {
        rows = m;
        cols = n;
        remaining = m * n;
    }

    public int[] flip() {
        int pick = rand.nextInt(remaining--);
        int actual = map.getOrDefault(pick, pick);
        map.put(pick, map.getOrDefault(remaining, remaining));  // ← map pick → current tail
        return new int[] {actual / cols, actual % cols};
    }

    public void reset() {
        remaining = rows * cols;
        map.clear();
    }
}
```

Time O(1) flip, O(1) reset | **Space** O(flips between resets)

#622 Design Circular Queue

Description: Design a fixed-capacity circular queue with `enQueue`, `deQueue`, `Front`, `Rear`, `isEmpty`, `isFull`.

Algorithm: Backing array with a `head` index and a `count`. Compute positions modulo capacity; the tail is `(head + count − 1) % capacity`.

Intuition: tracking `head` and `count` (rather than head/tail pointers) avoids the empty-vs-full ambiguity entirely.

```
class MyCircularQueue {
    private final int[] data;
    private final int capacity;
    private int head = 0, count = 0;

    public MyCircularQueue(int k) {
        capacity = k;
        data = new int[k];
    }

    public boolean enQueue(int value) {
        if (isFull()) {
            return false;
        }
        data[(head + count) % capacity] = value;
        count++;
        return true;
    }

    public boolean deQueue() {
        if (isEmpty()) {
            return false;
        }
        head = (head + 1) % capacity;
        count--;
        return true;
    }

    public int Front() {
        return isEmpty() ? -1 : data[head];
    }

    public int Rear() {
        return isEmpty() ? -1 : data[(head + count - 1) % capacity];
    }

    public boolean isEmpty() {
        return count == 0;
    }

    public boolean isFull() {
        return count == capacity;
    }
}
```

Time O(1) all ops | **Space** O(k)

#641 Design Circular Deque

Description: Design a double-ended circular queue with `insertFront`, `insertLast`, `deleteFront`, `deleteLast`, `getFront`, `getRear`, `isEmpty`, `isFull`.

Algorithm: Array with `head` index and `count`. Front insert moves `head` backward modulo capacity; rear insert writes at `(head + count) % capacity`. ← VARIATION: front insert decrements `head`.

Intuition: the #622 head/count model extends to both ends — inserting at the front just rotates `head` backward.

```
class MyCircularDeque {
    private final int[] data;
    private final int capacity;
    private int head = 0, count = 0;

    public MyCircularDeque(int k) {
        capacity = k;
        data = new int[k];
    }

    public boolean insertFront(int value) {
        if (isFull()) {
            return false;
        }
        head = (head - 1 + capacity) % capacity;   // ← VARIATION: rotate head backward
        data[head] = value;
        count++;
        return true;
    }

    public boolean insertLast(int value) {
        if (isFull()) {
            return false;
        }
        data[(head + count) % capacity] = value;
        count++;
        return true;
    }

    public boolean deleteFront() {
        if (isEmpty()) {
            return false;
        }
        head = (head + 1) % capacity;
        count--;
        return true;
    }

    public boolean deleteLast() {
        if (isEmpty()) {
            return false;
        }
        count--;
        return true;
    }

    public int getFront() {
        return isEmpty() ? -1 : data[head];
    }

    public int getRear() {
        return isEmpty() ? -1 : data[(head + count - 1) % capacity];
    }

    public boolean isEmpty() {
        return count == 0;
    }

    public boolean isFull() {
        return count == capacity;
    }
}
```

Time O(1) all ops | **Space** O(k)

#705 Design HashSet

Description: Design a HashSet without built-in hash libraries, supporting `add`, `remove`, and `contains`.

Algorithm: Bucket array with separate chaining — each bucket is a linked list of keys. Hash by modulo of a fixed bucket count.

Intuition: separate chaining is the textbook collision strategy; a fixed prime-ish bucket count keeps chains short for typical inputs.

```
class MyHashSet {
    private final List<Integer>[] buckets;
    private static final int SIZE = 1009;

    @SuppressWarnings("unchecked")
    public MyHashSet() {
        buckets = new LinkedList[SIZE];
        for (int i = 0; i < SIZE; i++) {
            buckets[i] = new LinkedList<>();
        }
    }

    public void add(int key) {
        List<Integer> bucket = buckets[hash(key)];
        if (!bucket.contains(key)) {
            bucket.add(key);
        }
    }

    public void remove(int key) {
        buckets[hash(key)].remove(Integer.valueOf(key));
    }

    public boolean contains(int key) {
        return buckets[hash(key)].contains(key);
    }

    private int hash(int key) {
        return Math.floorMod(key, SIZE);
    }
}
```

Time O(1) average per op | **Space** O(n)

#706 Design HashMap

Description: Design a HashMap without built-in hash libraries, supporting `put`, `get`, and `remove`.

Algorithm: Bucket array with separate chaining where each bucket stores key-value pairs. On `put`, update an existing pair or append; `get` scans the bucket; `remove` deletes the matching pair. ← VARIATION: store key-value entries instead of bare keys.

Intuition: identical structure to #705 but each node carries a value, so collisions are resolved by scanning a short chain of entries.

```
class MyHashMap {
    private final List<int[]>[] buckets;
    private static final int SIZE = 1009;

    @SuppressWarnings("unchecked")
    public MyHashMap() {
        buckets = new LinkedList[SIZE];
        for (int i = 0; i < SIZE; i++) {
            buckets[i] = new LinkedList<>();
        }
    }

    public void put(int key, int value) {
        List<int[]> bucket = buckets[hash(key)];
        for (int[] entry : bucket) {
            if (entry[0] == key) {           // ← VARIATION: update existing value
                entry[1] = value;
                return;
            }
        }
        bucket.add(new int[] {key, value});
    }

    public int get(int key) {
        for (int[] entry : buckets[hash(key)]) {
            if (entry[0] == key) {
                return entry[1];
            }
        }
        return -1;
    }

    public void remove(int key) {
        List<int[]> bucket = buckets[hash(key)];
        Iterator<int[]> iterator = bucket.iterator();
        while (iterator.hasNext()) {
            if (iterator.next()[0] == key) {
                iterator.remove();
                return;
            }
        }
    }

    private int hash(int key) {
        return Math.floorMod(key, SIZE);
    }
}
```

Time O(1) average per op | **Space** O(n)

#707 Design Linked List

Description: Design a linked list supporting `get`, `addAtHead`, `addAtTail`, `addAtIndex`, and `deleteAtIndex`.

Algorithm: Doubly linked list with head and tail sentinels and a size counter. Walk to the target index, then splice nodes by relinking `previous` / `next`.

Intuition: sentinels remove all null-edge cases for head/tail insertion, so every operation is the same splice on interior nodes.

```
class MyLinkedList {
    private final Node head = new Node(0);
    private final Node tail = new Node(0);
    private int size = 0;

    public MyLinkedList() {
        head.next = tail;
        tail.previous = head;
    }

    public int get(int index) {
        if (index < 0 || index >= size) {
            return -1;
        }
        Node current = head.next;
        for (int i = 0; i < index; i++) {
            current = current.next;
        }
        return current.val;
    }

    public void addAtHead(int val) {
        insertAfter(head, val);
    }

    public void addAtTail(int val) {
        insertAfter(tail.previous, val);
    }

    public void addAtIndex(int index, int val) {
        if (index < 0 || index > size) {
            return;
        }
        Node previous = head;
        for (int i = 0; i < index; i++) {
            previous = previous.next;
        }
        insertAfter(previous, val);
    }

    public void deleteAtIndex(int index) {
        if (index < 0 || index >= size) {
            return;
        }
        Node current = head.next;
        for (int i = 0; i < index; i++) {
            current = current.next;
        }
        current.previous.next = current.next;
        current.next.previous = current.previous;
        size--;
    }

    private void insertAfter(Node previous, int val) {
        Node node = new Node(val);
        node.next = previous.next;
        node.previous = previous;
        previous.next.previous = node;
        previous.next = node;
        size++;
    }

    static class Node {
        int val;
        Node previous, next;
        Node(int val) { this.val = val; }
```

```
    }
}
```

Time O(index) get/add/delete | **Space** O(n)

#729 My Calendar I

Description: Implement `book(start, end)` for a half-open interval `[start, end)`; return true and add the booking if it does not overlap any existing one, else false.

Algorithm: Keep bookings in a TreeMap keyed by start. For a new event, examine the floor and ceiling entries — an overlap exists if the previous event ends after `start` or the next event starts before `end`.

Intuition: the sorted map narrows the conflict check to at most two neighbors, giving O(log n) per booking.

```
class MyCalendar {
    private final TreeMap<Integer, Integer> calendar = new TreeMap<>();

    public boolean book(int start, int end) {
        Integer previous = calendar.floorKey(start);
        Integer next = calendar.ceilingKey(start);
        if (previous != null && calendar.get(previous) > start) {
            return false;
        }
        if (next != null && next < end) {
            return false;
        }
        calendar.put(start, end);
        return true;
    }
}
```

Time O(log n) per book | **Space** O(n)

#731 My Calendar II

Description: Implement `book(start, end)`; return true unless adding it would cause a triple booking (three events overlapping at the same instant).

Algorithm: Sweep-line with a TreeMap of delta counts: +1 at `start`, -1 at `end`. After tentatively adding the deltas, scan prefix sums; if any exceeds 2, roll back. ← VARIATION: allow double but reject triple overlap via running count.

Intuition: the difference array turns "max concurrent events" into a prefix-sum scan, and rolling back keeps the structure clean on rejection.

```
class MyCalendarTwo {
    private final TreeMap<Integer, Integer> delta = new TreeMap<>();

    public boolean book(int start, int end) {
        delta.merge(start, 1, Integer::sum);
        delta.merge(end, -1, Integer::sum);
        int active = 0;
        for (int value : delta.values()) {
            active += value;
            if (active > 2) {                  // ← VARIATION: triple booking detected, roll back
                delta.merge(start, -1, Integer::sum);
                delta.merge(end, 1, Integer::sum);
                return false;
            }
        }
        return true;
    }
}
```

Time O(n) per book | **Space** O(n)

#732 My Calendar III

Description: Implement `book(start, end)` returning the maximum `k` such that some instant is covered by `k` events (the largest k-booking so far).

Algorithm: Sweep-line delta map (+1 start, -1 end). After each booking, scan prefix sums and return the running maximum of concurrent events. ← VARIATION: return max overlap instead of a boolean.

Intuition: the same difference-array sweep as #731, but here we report the peak concurrency rather than reject it.

```
class MyCalendarThree {
    private final TreeMap<Integer, Integer> delta = new TreeMap<>();

    public int book(int startTime, int endTime) {
        delta.merge(startTime, 1, Integer::sum);
        delta.merge(endTime, -1, Integer::sum);
        int active = 0, result = 0;
        for (int value : delta.values()) {
            active += value;
            result = Math.max(result, active);   // ← VARIATION: track peak overlap
        }
        return result;
    }
}
```

Time O(n) per book | **Space** O(n)

#933 Number of Recent Calls

Description: Implement `ping(t)` (calls arrive with strictly increasing `t`) returning how many pings occurred in the inclusive window `[t - 3000, t]`.

Algorithm: Keep a queue of timestamps. On each ping, enqueue `t` and dequeue any timestamp older than `t - 3000`; the remaining queue size is the answer.

Intuition: because times arrive sorted, expired pings are always at the front — a sliding window over a queue.

```
class RecentCounter {
    private final Queue<Integer> queue = new ArrayDeque<>();

    public RecentCounter() {
    }

    public int ping(int t) {
        queue.offer(t);
        while (queue.peek() < t - 3000) {
            queue.poll();
        }
        return queue.size();
    }
}
```

Time O(1) amortized per ping | **Space** O(window size)

#1570 Dot Product of Two Sparse Vectors

Description: Design a `SparseVector` initialized from an array; `dotProduct(other)` returns the dot product, taking advantage of sparsity.

Algorithm: Store only nonzero entries as index→value pairs. For the dot product, iterate the smaller map and multiply where the other map has a matching index. ← VARIATION: iterate the smaller map for efficiency.

Intuition: since most entries are zero, storing and iterating only nonzeros makes the cost proportional to the number of nonzero terms, not the vector length.

```
class SparseVector {
    private final Map<Integer, Integer> map = new HashMap<>();

    SparseVector(int[] nums) {
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != 0) {
                map.put(i, nums[i]);
            }
        }
    }

    public int dotProduct(SparseVector vec) {
        Map<Integer, Integer> smaller = map;
        Map<Integer, Integer> larger = vec.map;
        if (smaller.size() > larger.size()) {       // ← VARIATION: iterate fewer entries
            Map<Integer, Integer> temp = smaller;
            smaller = larger;
            larger = temp;
        }
        int result = 0;
        for (Map.Entry<Integer, Integer> entry : smaller.entrySet()) {
            Integer other = larger.get(entry.getKey());
            if (other != null) {
                result += entry.getValue() * other;
            }
        }
        return result;
    }
}
```

Time O(n) init, O(min nonzeros) per dot product | **Space** O(nonzeros)